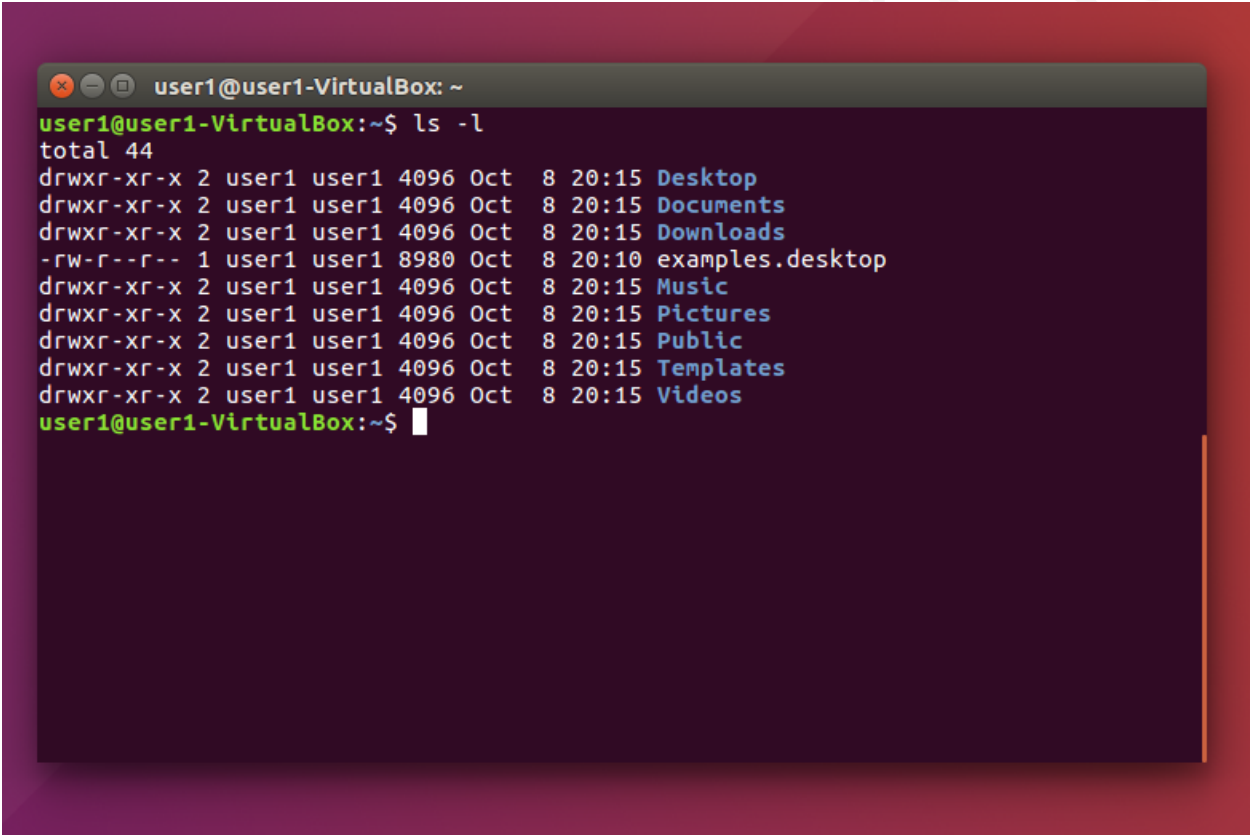


Linux Permissions Explained

By Caitlyn "CJ" Johnson

When a user creates a file, they own it. They have executive permission to that creation. They can create and modify and delete as they want. They are the "su" (aka the administrator) of their creation. Creators give and change permissions for others – the people using your creation.

Users, Groups and Others all have different permissions. Every single file and directory and link and shortcut and device.... EVERYTHING is *technically* a file in Linux and they all need permissions to function. The permissions can be found in several places, but in the terminal, you can type "ls -l" to see the long list of the items in your current directory. The permissions are on the left.



```
user1@user1-VirtualBox: ~  
user1@user1-VirtualBox:~$ ls -l  
total 44  
drwxr-xr-x 2 user1 user1 4096 Oct  8 20:15 Desktop  
drwxr-xr-x 2 user1 user1 4096 Oct  8 20:15 Documents  
drwxr-xr-x 2 user1 user1 4096 Oct  8 20:15 Downloads  
-rw-r--r-- 1 user1 user1 8980 Oct  8 20:10 examples.desktop  
drwxr-xr-x 2 user1 user1 4096 Oct  8 20:15 Music  
drwxr-xr-x 2 user1 user1 4096 Oct  8 20:15 Pictures  
drwxr-xr-x 2 user1 user1 4096 Oct  8 20:15 Public  
drwxr-xr-x 2 user1 user1 4096 Oct  8 20:15 Templates  
drwxr-xr-x 2 user1 user1 4096 Oct  8 20:15 Videos  
user1@user1-VirtualBox:~$
```

Regardless of what kind of file or directory you're dealing with, there are 10 "-" associated with your item. The first dash "-" helps define what kind of file it is. It could be a directory (d), or a block device aka a storage device (b), or a character device aka some type of hardware (c), or a soft link aka shortcut (l), or even a socket aka a device with input/output (*usually a network device*) (s). The first character lets the system define the type of thing. If there is no letter and it is just a dash "-", then it is a file. There are standard permissions for all these things that can be changed. Files are automatically -rw-r--r-- and directories are automatically drwxr-xr-x. You can change what the standards are using the umask command in the terminal, but that's more advanced than you'll need in almost all cases.

The next 9 dashes are UGO: permissions for the user, the group, and the others (everyone who does not fall in the first two sections). Each section, or octet, is composed of 3 dashes "-", no more or less. The user

section is first and dictates how the user can interact with the content, the group is the second three dashes and determines how the group can interact, and the everyone else is lumped together in the 3rd three spots.

---/---/---

All three sections are strung together with no actual break in the terminal, unlike the picture above. There are exactly 3 permissions you will see: read, write and execute. Read permissions (r) are to open and see, Write permissions (w) are to modify and edit, and Execute permissions are to run (an application or service or script). That is true for U, G, & O octets. The status of each permission can only be one of two things; on or off. If the permission is “on”, it is true – a user, group, or other could read, write or execute with the service. If a permission is “off”, then there is a dash “-” in the place of a letter. It is a light switch. Is the light on or off? If a letter is not there, the permission does not apply. This is all related to the concept of binary numbers. Computers love binary. Zeros are off and Ones are on in binary. These permissions work the exact same way.

rwXrwxrwx

Now that we’ve established the 3 permissions you can have, you need to understand that each permission has a numeric value. Computer systems love numbers and patterns. Linux operating systems are not special; they like to calculate and find it easy to count. The numbers are weighed the same in every octet (one section does not count more or less than any other). The numbers go from highest to lowest. Every “r” permission in each octet is worth four (4). The “w” permission is the same as the number two (2). Execute (x) permissions are always one (1). Those numbers are the same as the letter. Why those 3 numbers? The internet says it’s related to UNIX and bit marks... the why does not change the fact that these numbers are always true. With that in mind, your permissions – each octet – will always be between zero (0) and seven (7). Always. Each part of UGO cannot exceed 7. Why? Because $4 + 2 + 1 = 7$. Those are the only 3 numbers we know related to permissions. If there is a dash “-” and not a four (4), two (2) or one (1) in each permission octet, it means the permission is off. The permission does not apply. The computer sees nothing there and skips to the next piece of information.

r = 4, w = 2, x = 1

To show you how this works, I have made an example file named “example.txt”. I have not altered any standards, so its permissions will be -rw-rw-r--, as we discussed previously.

```
user1@user1-VirtualBox:~$ ls -l example.txt
-rw-rw-r-- 1 user1 user1 0 Nov  7 21:01 example.txt
user1@user1-VirtualBox:~$
```

This means, in numbers, the file has 644 as its permissions. Let’s break it down;

The first octet is the User. It is showing as the number 6. So, we have to ask three questions to confirm the permission. Is the read permission on? Yes. Is the write permission on? Yes. Is the execute permission on? No. Let’s ask those same questions with the numbers instead. Is the 4 permission on? Yes, because r IS 4. Is the 2 permission on? Yes, because w IS 2. Is the execute permission on? No, there is no x in the first section, just a dash. The “x” permission is off. The computer would see zero in its place. $4 + 2 + 0 = 6$. We repeat this process and ask the same questions about the Group octet and the Others octet. Four (4) is the only number represented. It means NO other number can be applied or found. It means the write and execute permissions MUST be off. For both the Group and the Others octets; $4 + 0 + 0 = 4$. It is simple addition that can never exceed the number seven.

644; -420400400 (-rw-r--r--)

Math never changes. Here are some examples, with the math below the permission string. Also, as a reminder the FIRST dash “-” is defines what kind of item it is.... There is no number listed. A simple dash “-” at the beginning just means that we’re working with a file.

777; -421421421 (-rwxrwxrwx)

$$4+2+1 = 7, 4+2+1 = 7, 4+2+1 = 7$$

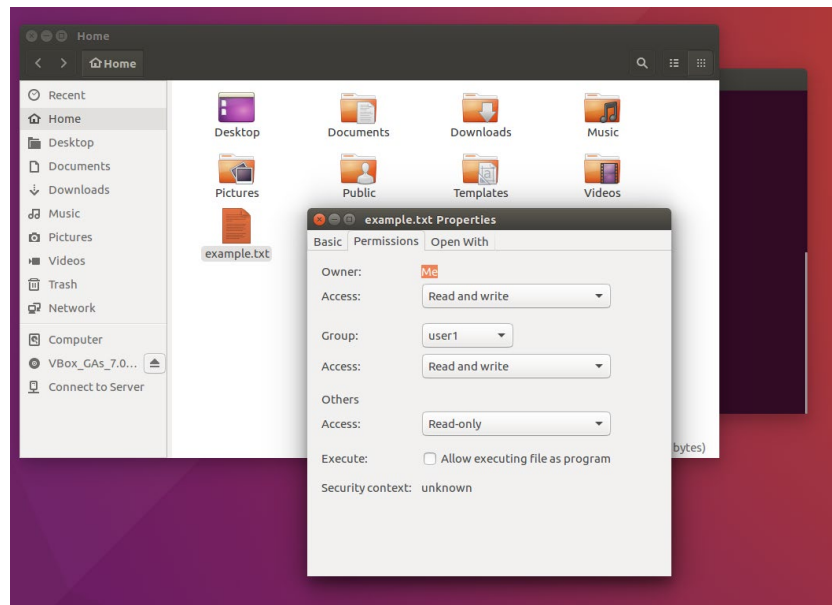
532; -401021020 (-r-x-wx-w-)

$$4+0+1 = 5, 0+2+1 = 3, 0+2+0 = 2$$

110; -001001000 (---x--x---)

$$0+0+1 = 1, 0+0+1 = 1, 0+0+0 = 0$$

As a reminder, you can also change the permissions in the GUI. If you look at any file, you can go to the properties and see/change the permissions.



Now that we have a basic understanding (hopefully?) of permissions and numbers, let's talk about changing them. If you want to change the permissions in the terminal (which is both simple and fast. It is also where the LPI certification will expect you to do this), use the command "chmod". Think of it as "change/modify" permissions. You do not need to be a super user or administrator to use this command. You will use this command in the same order that you would do any other (command, options, arguments); `chmod UGO + OR - OR = rwx OR 777 file name`. Let's make that last sentence less confusing.

```
-rw-rw-r-- 1 user1 user1 0 Nov 7 21:01 example.txt
user1@user1-VirtualBox:~$ chmod ugo+x example.txt
user1@user1-VirtualBox:~$ ls -l example.txt
-rwxrwxr-x 1 user1 user1 0 Nov 7 21:01 example.txt
```

In the screenshot, we can see we added the execute permission ("x" aka 1) to the User, Group AND Others octets. We can then see it worked! We know this because the color changed, but more importantly the permissions all have an additional x displayed instead of a dash "-". You can change the permissions to each section together or separately. You can also use letters or numbers to change permissions. You do NOT need to use symbols (+, -, =) if you use numbers (the math is implied).

```
user1@user1-VirtualBox:~$ chmod 654 example.txt | ls -l example.txt
-rw-r-xr-- 1 user1 user1 0 Nov 7 21:01 example.txt
user1@user1-VirtualBox:~$
```

Above, we can see I changed the permissions to the User, Group and Others collectively using the `chmod` command and the numbers 654 (rw-r-xr--). I also used a pipe with the `ls` command, allowing us to see the results of the change immediately (because pipes redirect the output of the 1st command to the 2nd command). Again, if you wanted to, you could change the Group permission by typing `chmod g-x example.txt` and remove the Group's ability to execute the file. Because there are no "x" in the User, Group, or Others octet, the color of the file changes back to white (white is files, blue is directories).

```
user1@user1-VirtualBox:~$ chmod g-x example.txt
user1@user1-VirtualBox:~$ ls -l example.txt
-rw-r--r-- 1 user1 user1 0 Nov  7 21:01 example.txt
```

Try it! Touch a file! Touch a folder (directory)! *[make the name easy/fast to type repeatedly.]* Change the permissions using numbers and letters! Using both will work the same! You can use any numbers up to 7 (the math cannot work if it's 8 or more)! You can take all the permissions away from everyone.... If you do, then no one can touch it. Fun!

Now that we've covered the fundamentals of basic permissions, I am going to add one more layer and confuse you further.

As the creator of a thing in Linux, I give permission to others. I am the owner and I have rights. As the owner, I can do whatever I want with what I create. It's mine. I can move it or share it or delete it. Other people may want or need my rights to something. I can share that right, not just give permissions. If there is a SUID or SGID bit attached to permissions, someone else can act like they're the great and powerful creator. Those fun 4 letters mean you want to **Set The User ID** and **Set The Group ID**. Why would you do this? Well, you may need to share different files and projects. You could need to run a script, but it isn't your file to alter and add execute "x" permissions.... Root can alter nearly everything, but we're not all root. The owner is like root for their own possessions. So being the owner has power.

The SUID and SGID have their own additional math. You will see this as a 4th octet at the beginning of the permissions with numbers. The permissions, in numbers, would look like this: 7777. Well, how did we get to that new seven (7) at the beginning? What does it even mean? More math that ONLY applies to the first number in an octet of four.

4 = SUID is on, 2 = SGID is on, 1 = a Sticky bit has been applied.

So, let's look further (don't give up now). If there is a 4 at the beginning of the permissions string, the User could act as the owner, so you need to set the user id (SUID). If there is a 2 at the beginning of the permissions string, that means the Group can act as the owner so the Set Group ID needs to be on. If there is a 1 at the beginning of the permissions, before the regular UGO permissions, it means that there is an active sticky bit. A sticky bit is a note to the system that only the owner can delete their own property. It means someone else cannot change the name or get rid of your files and folders. If you're sharing something, mistakes can happen. Sticky bits mean that you own it, and I can't mess with it.... Unless I'm root. If there is a 7 at the beginning of the string, it means all 3 super special permissions are on – the User ID and the Group ID and the Sticky Bit are all on. You can mix and match them, depending on who is trying to do what. The additional first number can not exceed seven (7) $[4+2+1 = 7]$. Think of this additional number as an elevated status... you are trying to be special User or a super Group and Others cannot mess with your stuff. Let's get more ridiculous switching to letters.

With letter permissions, you will see an "s" in the place of the "x" permission to notify of the SUID and/or SGID being on. So, a 7777 string would look like this;

7777; rwsrwsrwt

What the what? “s” is there now?! Let’s dive deeper into it. An “s” just means that the special elevation is on for the User ID, Group ID, and the “t” is the sticky bit. The main rwx permissions are still applied. So, if there was a 4777 (rwsrwxrwx) or a 2777 (rwxrwsrwx) or a 1777 (rwxrwxrwt), we’re just turning octets on and off as a user being the owner. We are taking extra control! If there is an additional number at the beginning of the permissions, it means; SUID/GUID/Sticky Bit UGO. It means rights supersede privileges for at least one of the UGO. You, depending on which “s” is on (which/where the elevation of status is), you can change permissions or delete things or change and modify and send the directory to the moon. The item believes you are the owner even if you are not because the “s” says so.

6754; -rwsr-sr--

This means the User has and Group can act as the owner; others can only read the file.

5632; -rwS-wx-wT

This means that the User has elevated permissions and the sticky bit has been set.

7444; r-Sr-Sr-T

This means that the User and Group are special and the sticky bit has been set.

Now, if you noticed above, sometimes I use a capital “S” and others I use a lowercase “s”. I also used the letter “T”. You might think my syntax is off. But there’s a little more. If you see a capital letter (so “S” or “T”), the execute permission is off for everyone else. You’re acting as the owner. You can do whatever you want as the owner. We know this. But the UGO permissions don’t go away and the “s” and “t” to identify have to go somewhere in the permission string. There are only 9 dashes “-” to work with. So, acting as the owner with the SUID or SGID, we display with the “s” but the UGO need to let others (*and the system*) know what the actual permissions (not the rights you’re using). Is execute on or off with SUID and SGID? Capital letters mean it’s OFF, but you are elevated. S = you’re special but execute permission is off in User or Group. s = execute is on AND you’re acting as owner in User and Group. T means the sticky bit is set but Others can’t execute the file. t = the sticky bit is on and Others can execute too.

***I know it can be confusing. Do research. Do not panic. You’re just making sentences in the terminal.
You’re just telling the computer who can do what and how they can do it.***

SUMMARY:

UGO = User, Group, Others permissions on everything in Linux OS'.

rwX = read, write and execute permissions, which represent the numbers 4,2,1.

Permissions, when displayed in numbers cannot exceed 7 in each section of 3 (rwX is 421) for each section (UGO). Letters must be “on” (a letter displayed) or “off” (a dash “-”). Each octet has it's own rwX spots.

You can change the permissions with the chmod command in the terminal.

The SUID and SGID & Sticky Bit allow you to act as the owner with a file. An owner has rights and gives permissions. Regular users use the permissions set. If there is a 4th number added to the octet, or an “s” or “t” sitting in the “X” permission spot (the 3rd dash“-” in each octet), then you can act as the owner and manipulate the file however you'd like.

Happy Scripting. Please ask for help. From me. From AI. From Google. From Flex. From a Friend. Ask someone. Do not stay confused.